XCEL UX DASHBOARD · FOR DESIGNERS

Designer workflow on the Hub

From a laptop with nothing on it to a branch under review on the Hub. Every step is a sentence you type to Claude. You never open a terminal, and you never need to know a git command. When a step shows a sentence in quotes, that is exactly what you type.

Last updated 15 October 2026 · Owner: Jillienne Moreno · Applies to `jillmoreno/xcel-ux-dashboard`

CONTENTS

1. Before you start
2. One-time setup
3. Every time you sit down
4. Eric: rebase your branch
5. Anjani: start fresh
6. Building a change, start to finish
7. Sharing on Refinement
8. Making your surface the prototype
9. Reading the Hub
10. When something goes wrong
11. The routines
12. The morning auto-rebase
13. Glossary

01 Before you start

There are three places, and it helps to know which one you are looking at:

PLACE	WHAT IT IS	WHERE
GitHub	The one shared copy of the code. Every branch lives here.	`github.com/jillmoreno/xcel-ux-dashboard`
Netlify	Builds every branch into a website at its own address, a few minutes after each push.	`feat-your-branch--ux-design-xceldashboard.netlify.app`
The Hub	The UX Design Hub: Prototypes, Refinement, Explorations and the rest. One site, one password.	`ux-design-xceldashboard.netlify.app`

Six words you will see over and over:

WORD	PLAIN MEANING
`main`	The one branch that IS the product. Only Jillienne changes it.
branch	Your own copy of the code, named like `feat/compass-week-view`. Everything you do happens on a branch.
commit	A saved snapshot of your branch with a short note saying what changed.
push	Sending your commits to GitHub. Netlify then builds them and the team can see them.
rebase	Moving your branch so it starts from today's `main` instead of the day you created it.
merge	Jillienne putting your branch into `main`. She calls this "ship to main".

THE ONE RULE UNDER EVERYTHING

You work on your branch, Jillienne works on `main`. If you ever find yourself on `main`, stop and tell her.

02 One-time setup

Do this once per computer. It takes about twenty minutes, most of it waiting.

1 Install the Claude desktop app

Download it from claude.ai and sign in with the account Jillienne invited. Claude Code is the **Code** tab inside that app. It is a chat window that can also read and change files on your computer.

2 Make an empty folder

On your computer, called `xcel`. Documents is fine.

3 Open the Code tab

It asks which folder to work in. Pick the `xcel` folder you just made.

4 Get the code

In the chat box, type: *"Clone https://github.com/jillmoreno/xcel-ux-dashboard into this folder."* Claude will show you the command it wants to run and ask for permission. Click **Allow**. If a browser window opens asking you to sign in to GitHub, sign in as yourself (Anjani is `Anjanisatya5`, Eric is `EnormanColibri`). You are both already collaborators on the repository.

5 Point Claude at the project

The clone made a folder inside `xcel` called `xcel-ux-dashboard`. Use the Code tab's folder picker to open that inner folder. From now on, every session starts by opening this folder.

6 Install what the project needs

Type: *"Install the project's dependencies."* Allow it. This downloads about a minute's worth of packages. If Claude says Node or npm is not installed, type: *"Install Node LTS for me, then install the dependencies."* Allow each step.

7 Put your name on your work

Type, with your own details: *"For this project, set my git name to Anjani Satya and my git email to the address on my GitHub account."* Eric: "Eric Norman" and `eric.norman@simplenursing.com`.

This matters more than it looks. The Hub decides who owns a change by who authored it, and Anjani's earlier commits were recorded as written by Claude because this was never set.

8 Check it

Type: *"Which branch am I on, and what are my git name and email?"* You should hear `main` and your own name. That is the only time being on `main` is fine, because you have not changed anything yet.

You do not install Netlify, Vite, or anything else by hand. You do not run the site on your own computer. The branch build on Netlify is your preview.

03 Every time you sit down

Opening up. Open the Claude app, go to the Code tab, and make sure the folder shown is `xcel-ux-dashboard`. Start a new conversation for each piece of work. Old conversations are fine to leave.

How to talk to it.

1. Type plain sentences, one step at a time. *"Rebase my branch onto main"* is better than three requests in one message.
2. Claude tells you what it is about to do before it does it. Read that line.
3. Before it runs a command or changes a file, a permission card appears with **Allow** and **Deny**. Click Allow when the card describes the thing you asked for. If it describes something you did not ask for, click Deny and ask what it was trying to do.
4. If Claude refuses to edit a file and names it, that refusal is correct. Those files are the Hub's own machinery and only Jillienne changes them. Tell her what you needed.
5. Typing `/` in the chat box shows the project's ready-made routines. The ones you will use are `/promote-component`, `/share` and `/promote-to-prototype`. Each one asks you questions and does the steps. Section 11 says what each one asks, what it does, and when to reach for it.
6. When Claude finishes, it summarises what it did. If anything in that summary surprises you, ask before going on.

The three questions to ask before touching anything.

ASK	WHAT A GOOD ANSWER SOUNDS LIKE	IF NOT
"Which branch am I on, and do I have uncommitted changes?"	Your `feat/…` branch, and either "no changes" or a list you recognise.	On `main`: stop, tell Jillienne. Changes you don't recognise: ask Claude what they are before anything else.
"Fetch origin. How far behind main is my branch?"	A number of commits. Under ten is fine for now.	Ten or more: rebase before building anything new (section 04 shows how).
"Has anything on main changed the files I'm working on?"	"No", or a short list.	A list: read it, because that is where a conflict would come from later.

Rebasing every few days is painless. Rebasing once at the end of a month is not.

04 Eric: rebase `feat/atlas-compass-global-nav`

> **A SNAPSHOT, NOT A STANDING INSTRUCTION**
>
> The numbers in this section were true on the evening of 15 October 2026. The ten steps are the general recipe and stay right; the counts will not. Re-ask Claude *"how far behind main is my branch?"* rather than trusting the figures below.

As of that evening your branch has 10 commits that are not on `main` (the checkout payment rework, the product-detail rail, the Home styling, the password walk-through), and `main` has moved 74 commits past where you started. Four of your ten already reached `main` that afternoon in another

form, through Jillienne's invite promotion; git will recognise those and drop them during the rebase, which is normal. Those 67 include the one-site change, the new Hub nav, and Anjani's second Compass master. Your branch needs to be moved on top of all that before anything else happens to it.

A rebase does not change your work. It replays your commits, one at a time, on top of today's `main`, as if you had started this morning. Do it one sentence at a time and wait for each answer.

1 Confirm where you are

"Which branch am I on, and do I have uncommitted changes?" You want to hear `feat/atlas-compass-global-nav`. If you are somewhere else: *"Switch to feat/atlas-compass-global-nav."*

2 Save anything loose

If step 1 listed changes: *"Commit everything on this branch with a message that says what it is."* A rebase moves commits; it cannot move work you have not committed.

3 Make a safety copy

"Create a backup branch called backup/atlas-compass-global-nav-oct-15 pointing at where I am now, and push it." Now nothing you do next can lose work. You can delete it next week.

4 Do the rebase

"Fetch origin and rebase this branch onto origin/main. If anything conflicts, stop and show me both sides before changing anything." Allow it.

Expect it to stop at least once this time: `main` took your morning's Confirmation page and added a flag to it (the flow now ends at a loading page), and your afternoon commits edit the same three files — `InvitedCheckoutPage`, `InvitedConfirmationPage`, `InvitedProductDetailPage`. That is exactly the case step 5 is for.

5 If it stops on a conflict

Claude will say a file name and show two versions of the same lines: what `main` has, and what your commit has. Read both. Then say one of: *"Keep my version"*, *"Keep main's version"*, or *"Keep both — main's block first, then mine."* Then: *"Continue the rebase."* It may stop again on the next commit; do the same thing.

If you cannot tell which to keep, say *"Abort the rebase"* — your branch goes back exactly as it was — and ask Jillienne.

6 Confirm it worked

"How far behind main is my branch now, and how many commits ahead?" You want: 0 behind, and about 5 or 6 ahead (the four commits `main` already had will have dropped out).

7 Check the build

"Run the build, the tests, and lint on the files I changed." The build is the gate Netlify uses; if it fails here, your branch will not deploy. If something fails, read Claude's summary and say *"Fix that and run it again."* If it is in a file you never touched, stop and ask Jillienne.

8 Push

"Push my rebased branch. It will need a force push with lease." A rebase rewrites your branch's history, so a normal push is refused with a message about the branch being behind. Force-with-lease is the safe form: it refuses if someone else pushed to your branch in the meantime. This is normal on your own branch and is never done on `main`.

9 Get the link

"Give me the Netlify review link for this branch at the invite flow." It is `https://feat-atlas-compass-global-nav--ux-design-xceldashboard.netlify.app/end-to-end-demo?demo=1`. Netlify starts building the moment you push and takes two to four minutes. The page asks for the same password as the Hub.

10 **Look at it**

Click through the invite flow from the start, the way a reviewer would. If it looks like it did before the rebase, you are done. If something that used to work is broken, that is where `main` and your branch disagree; tell Jillienne which page.

From here your branch is current. Rebase again every few days with steps 4, 7 and 8; it will be quicker each time. Or let Claude do it for you every morning (section 12).

05 Anjani: your master is on `main`; start fresh

> A SNAPSHOT, NOT A STANDING INSTRUCTION
>
> True as of 15 October 2026. The seven steps are the general recipe; the state described in the first paragraph is not.

You have nothing to rebase. Your second Compass master — the rail, Learn, the week on the Overview, the planner's three concepts — was ported into the React app and shipped to `main` that afternoon. Open the Hub, go to Prototypes, and the Compass Learning row is your work at `main`'s current build. Your old branch from the first pull request (`claude/gallant-feynman-aj0m0i`) is finished; it is 355 commits behind and should not be reused.

Two things changed about how your work gets in:

- **Your Compass surface is React now, not HTML.** It lives in files like `src/components/learning/CompassRail.tsx`, `CompassLearningOverview.tsx`, `CompassStudyPlanner.tsx` and the stylesheet `compassAnjani.css`. When you want to change how Compass looks or behaves, the change goes there, on a branch, behind a flag. Claude knows these files; describe the change and it will find them.
- **A new HTML prototype is an exploration, not the product.** If you build a standalone HTML page, it goes under Explorations on the Hub, and adding it there means editing one of Jillienne's files. Send her the file and she adds the row. Do not try to add it to Prototypes; that section is `main`'s build only.

Starting your next piece of work:

1. Open the Code tab on `xcel-ux-dashboard`.
2. *"Which branch am I on, and do I have uncommitted changes?"* You want `main` with no changes, or an old branch with nothing loose on it.
3. *"Fetch origin and make sure my main matches origin/main."* This pulls down today's shipped work, including your own master.

4. *"Make a new branch called feat/compass-‹what you are doing› from main and switch to it."* Lowercase, dashes, short. For example `feat/compass-week-view-spacing`.
5. Confirm: *"Which branch am I on?"* It should say the new name.
6. Describe the change in plain words. For example: *"On the Compass Overview, the week strip should sit above the planner, not below it. Put this behind a feature flag that is on by default on this branch."* Claude builds it, behind a flag, and tells you the flag's name.
7. Carry on with section 06 from step 5 — or just say `/share`, which is steps 5 to 9 in one command.

Your review link will be `https://feat-compass-‹name›--ux-design-xceldashboard.netlify.app/dashboard-rebrand?demo=1&section=compass`. That is the same address the Compass Prototypes row uses, on your branch instead of `main`.

06 Building a change, start to finish

This is the loop you will repeat for every piece of work. Each step is one sentence to Claude.

1 Start from `main`

"Fetch origin, then make a new branch called feat/‹short-name› from origin/main and switch to it." Never branch from an old branch of yours; it drags the old branch's history along.

2 Say what you want

Describe the change as you would to a developer sitting next to you. Name the page, the component, what it should do instead. Claude reads the code and proposes where the change goes. If it names a file you don't recognise, ask *"What is that file?"* before saying yes.

3 Changing something that already exists? Use a sibling copy

Say `/promote-component` and name the component. It copies `StudyPaceTile.tsx` to `StudyPaceTileV2.tsx`, leaves the original untouched, and adds a flag that picks between them. This is why your branch almost never conflicts with anyone else's: nobody edits the same file.

4 The flag is on by default on your branch

Claude does this when you say *"behind a feature flag that is on by default on this branch"*. It means your branch build shows your version, and Jillienne can decide at merge whether it ships. Ask Claude *"What is the flag called?"* and write it down; the review link can turn it off with `&ff=flag-name:off` if you want to show the before.

5 Check it

"Run the type check, the tests, and lint on the files I changed." If you added or edited anything under `public/prototypes/`, add *"and the smoke suites"*. Red means fix before pushing: *"Fix that and run it again."*

6 Commit

"Commit everything on this branch with a message that says what changed and why." Claude writes the message in the project's style and shows it to you first.

7 Push

"Push my branch to GitHub." The first push on a new branch also sets it up on GitHub; Claude handles that. From then on, every push is the same sentence.

8 Wait for Netlify

Two to four minutes. *"Check whether my branch has finished building on Netlify"* if you want to know rather than guess.

9 Get the link

"Give me the review link for this branch." The shape is `https://‹branch-with-dashes›--ux-design-xceldashboard.netlify.app` followed by the page: `/dashboard-rebrand?demo=1` for the dashboard, `/dashboard-rebrand?demo=1&section=compass` for Compass, `/end-to-end-demo?demo=1` for the invite flow. `?demo=1` turns on the demo controls bar.

10 Look at it the way a reviewer would

Click through from the page they would start on, not straight to your component. Three bugs in September were invisible on a direct link and obvious on the first click.

11 Keep going

Each further change is step 2, then `/share` again. Rebase every few days (section 04, steps 4 to 8) so the final merge is boring. When the next change is really a different piece of work, start a new branch for it rather than letting this one grow.

STEPS 5 TO 9 ARE ONE COMMAND

`/share` runs the checks, commits, pushes, waits for the build and hands you the link, then opens the Refinement form — section 07. The long way above is what it is doing, which is worth knowing for the day it stops halfway.

WHAT YOU NEVER DO

Run the site on your own computer, edit `main`, or edit the Hub's own files — Claude will refuse and name them. Ask Jillienne.

07 Sharing on Refinement

Refinement is the board where work under discussion lives. Anyone adds to it, nothing on it is the source of truth, it filters by your name, and it has an archive.

A row is one chunk of work, not a branch you live on. Since 15 October rows group under the **feature** they are chunks of, newest first. When a new chunk carries on from an older one, the older one folds under "1 earlier chunk" — still there, still clickable — so a feature keeps a history you can open in a review. One long-running branch gives you a single link and no history at all. That is the whole reason to work in small pieces: finish one, share it, start a new branch for the next.

Adding your branch, the quick way.

1. On your branch in the Code tab, type `/share`. It was called `/promote-to-refinement` until 15 October, and that name still works.
2. Claude asks once — checks, commit, push, open the form? — then runs the type check and the tests, commits anything uncommitted, pushes, waits for Netlify to build your branch, works out its

address and a title, and asks what feature this is a chunk of. Nothing reaches the board until you click Save.

3. In the form, **Added by** is a pick-list of the five of you. Pick your own name. The product (Atlas, Compass, Both) is already chosen from what your branch touched; change it if it is wrong. **Feature** is what this chunk is part of — start typing and the features already on the board are offered, so take one if it fits. **Replaces** appears only when that feature already has chunks, and defaults to your own newest one; its first option, *Nothing — these sit side by side*, is the right answer when two treatments should both stay up.
4. Click Save. Your row appears at the top of its feature group, showing your name, when it was last saved, and the product. Claude then offers to cut your next branch.

Adding it by hand. Open the Hub, click Refinement, click Add link, paste the review link from section 06, type a title and a one-line note on what to look at, pick your name, pick the product, name the feature, Save.

Finding things. The pills above the list are the five names: All, Jillienne, Anjani, Eric, Afsana, Cody. Click a name to see only that person's rows; click it again to clear. There is no product pill any more; the product is the tag on the row. Rows sit under their feature heading, so looking for a piece of work means looking for its name rather than its branch.

When a row's discussion is over.

YOU WANT TO	DO THIS	WHAT HAPPENS
Tidy it off the board but keep it	Click the row's ⋮, choose **Archive**	The row disappears from the board. It is stamped with when and by whom. Nothing is deleted.
See what was archived	Click **Show archive · N** at the top right	The board switches to the hidden rows. **Back to board** returns.
Bring one back	In the archive, click its ⋮, choose **Unarchive**	It is back on the board. Anyone can do this.
Delete a row that should never have existed	Click ⋮, choose **Remove**	Gone for good, after a confirmation.

Use Archive by default. Remove is for mistakes.

TWO THINGS TO KNOW

The **Updated** tag changes only when the row is saved, not when your branch changes; if you push a big change, open the row and Save it again so the team sees fresh activity.

And a Refinement row follows `main` underneath: if a component you point at changes on `main`, the row changes with it. Anything that must stay frozen, a user test for instance, is Jillienne's `/promote-to-testing`, not a Refinement row.

08 Making your surface the prototype, and who ships it

Prototypes is the one section that means "this is the product". It holds one row per surface, every row is `main`'s build, and each surface has an owner who decides what it should be:

SURFACE ON PROTOTYPES	OWNER	REVIEW ADDRESS
End-to-End User Experience	Jillienne	held back for now
Invite & Purchase	Afsana and Eric	`/end-to-end-demo?demo=1`
Atlas Learning	Eric	`/dashboard-rebrand?demo=1&section=dashboard`
Compass Learning	Anjani	`/dashboard-rebrand?demo=1&section=compass`
Admin Tool	Cody	placeholder until it is built

There are two separate decisions, made by two different people:

- **What the surface should be** is the owner's call, and the owner records it by running the promotion on their own branch.
- **When it reaches `main`** is Jillienne's call, and only she can do it.

When your branch is what your surface should become:

1. Make sure the branch is current (section 04, steps 4 to 8) and the review link looks right.
2. On your branch, type `/promote-to-prototype` and name your scope: "for Compass" (Anjani), "for Atlas Learning" or "for Invite & Purchase" (Eric).
3. Claude lists the flags your branch introduced and asks, for each, whether it should be the default. Say yes to the ones that are your design and no to anything experimental you are not ready to stand behind.

4. It commits those defaults on your branch. Nothing has reached a stakeholder yet; this is a decision written down, not a release.
5. *"Push my branch."*
6. Tell Jillienne, in words: *"My Compass branch feat/… is ready to ship."* Include the review link.
7. Jillienne runs `/ship-to-main`. It refuses a branch whose owner has not run step 2, so the order matters.
8. A few minutes after she ships, your surface's row on Prototypes is your work. Its Updated tag changes to that moment.
9. Archive your Refinement row (section 07), since the board is for things under discussion and this one is decided.
10. Start the next piece from a fresh branch off `main` (section 06, step 1). Do not keep working on the shipped branch.

If you own the surface, you can also decide something should NOT go in. Say so, archive the Refinement row, and tell Jillienne; she has a routine that records why a branch was tried and declined.

09 Reading the Hub

The Hub is at `ux-design-xceldashboard.netlify.app`, behind one password that Jillienne gives out. Everyone, stakeholders included, sees the same thing. Nothing inside is locked. Ten sections in three groups:

GROUP	SECTION	WHAT IT IS FOR	WHO ADDS TO IT
Demo	**Prototypes**	The product as it stands: one row per surface, all `main`'s build.	Owners promote; Jillienne merges.
Design & Research	**Refinement**	Chunks of work under discussion, grouped by the feature they belong to. As many as you like. Filter by name, archive when done.	Anyone, on the page.
Design & Research	Research	Research links and rationale.	Anyone, on the page.
Design & Research	Resources	Briefs, Figma files, references. Things that live elsewhere — including this guide.	Anyone, on the page.
Design & Research	PRDs	The product requirements the designs answer.	Anyone, on the page.
Design & Research	Explorations	Standalone HTML documents: outside products and ideas rebuilt on our tokens. Five today.	Jillienne adds the row; send her the file.
Dev Handoff	Development	Handoff notes for engineering. Empty right now.	Written with `/dev-handoff-notes`, which Jillienne runs with you.
Dev Handoff	Done	Finished and signed off.	Jillienne.
Dev Handoff	Archive	Components and sections removed from the project, with how to bring each back.	Jillienne.
Dev Handoff	QA Notes	Design-versus-build findings with captures.	Anyone, on the page.

What went away on 15 October, in case you look for it: the second "demo" website, the Public toggle on Refinement rows, the locked Designers group in the nav, the Design section, and the Atlas / Compass filter pills on Refinement. The old Exploration section is the Explorations row under PRDs.

10 When something goes wrong

None of these lose work. Find the message, read what it means, say the sentence.

YOU SEE	IT MEANS	SAY TO CLAUDE
A refusal naming a file, like "UxDashboardPage.tsx is protected"	That file is the Hub's own machinery. Only Jillienne edits it.	Nothing. Tell Jillienne what you needed changed.
"Updates were rejected because the tip of your current branch is behind"	You rebased, so your branch's history changed, and a normal push is refused.	*"Push with force-with-lease."* (Your branch only. Never `main`.)
"CONFLICT (content): Merge conflict in …"	`main` and your commit changed the same lines. The rebase paused for you to choose.	*"Show me both sides."* Then *"Keep my version"* / *"Keep main's version"* / *"Keep both"*, then *"Continue the rebase."* Lost? *"Abort the rebase"* and ask Jillienne.
"fatal: 'main' is already checked out at …"	Someone tried to switch to `main` in a	*"Switch back to my*

YOU SEE	IT MEANS	SAY TO CLAUDE
	folder where that is not allowed. You should not be switching to `main` anyway.	*feature branch."*
The build fails with lines starting `error TS`	A type error. Netlify would refuse to deploy this.	*"Fix the type error and run the build again."* If it is in a file you never touched, stop and ask Jillienne.
Tests fail, and the names mention `TestingGateway` on the first run only	A known slow first run when caches are cold.	*"Run the tests again."* If it fails twice, it is real.
The Netlify link asks for a password	Every branch build is behind the same password as the Hub.	Nothing. Enter the Hub password.
The Netlify link shows an old version, or "Page not found"	The build has not	*"Check the Netlify deploy*

YOU SEE	IT MEANS	SAY TO CLAUDE
	finished, or failed.	*status for my branch."* If it failed, *"Show me why"* — usually the same type error as above.
"You are in 'detached HEAD' state"	Your work has no branch under it. Nothing is lost yet.	*"Create a branch from here called feat/‹name› and switch to it."*
Claude says it cannot run a command or lacks permission	The permission card was denied, or needs approving.	Click Allow on the card, or *"Try again."*
Your Refinement row shows an old Updated time	The stamp changes when the row is saved, not when the branch changes.	Open the row on the Hub, click Save.
Your commits show the author as "Claude"	Your git name was never set on this computer.	Section 02, step 7. Then: *"Re-author my commits on this branch to me."*

YOU SEE	IT MEANS	SAY TO CLAUDE
You are not sure what Claude just did	Anything.	*"Summarise what you changed in the last step, file by file."* Then decide.

IF NONE OF THESE FIT

Do not try a second fix on top of the first. Copy the message, send it to Jillienne, and leave the folder as it is.

11 The routines, and when to use each

A routine is a checklist Claude runs with you. Type `/` and its name in the chat box. It asks you a few questions, does the steps in order, and stops short of anything it should not decide for you. Three of them are yours. Five belong to Jillienne; know their names so you recognise them when she mentions one.

Which one do I want?

YOU ARE THINKING	REACH FOR
"I want eyes on this one card, not the whole branch."	`/promote-component`
"I want the team to look at my branch."	`/share`
"This is what my surface should be. Ship it."	`/promote-to-prototype`, then tell Jillienne
"I'm done discussing this row."	Archive, on the row's ⋮ (no routine)
"We tried it and decided not to."	Tell Jillienne; she runs `/retire-from-refinement`
"I need a frozen link for a user test."	Tell Jillienne; she runs `/promote-to-testing`

Yours. Three routines, and the thing worth knowing about each is as much what it will *not* do as what it will.

/PROMOTE-COMPONENT

Use it when	One component on your branch wants feedback and the rest of the branch would distract from it.
It asks you	Which component (name it, or select it in the browser pane first). Which of its flag's variants to show side by side. A note telling the reviewer what to look at.
It does	Finds the feature flag that drives the component. If there is none, shows you the flag it would add — its name, label, and the one line it changes in your component — and waits for your yes. Checks the branch is pushed and built. Builds one review page with every variant stacked, each pinned. Opens Refinement's Add link form with it filled in.
It never does	Save the row — you click Add. Add a flag without showing you first. Share the whole branch.

/SHARE

Use it when	A chunk of work is ready for the team to look at.
It asks you	One yes, up front, covering the checks, the commit and the push. A title. A one-line note on what to look at. What feature this is a chunk of. Which product — it guesses from the files you touched, so Compass files mean Compass.
It does	Checks you are on a branch, not `main`. Says so if the branch has got long. Runs the type check and the tests. Commits anything uncommitted and pushes. Works out the Netlify address and waits for the build to finish. Opens Refinement's Add link form with everything filled in, your name included. Offers to cut your next branch.
It never does	Save the row — you click Add. Run on `main`. Commit or push without the yes it asked you for.

/PROMOTE-TO-PROTOTYPE	
Use it when	Your branch is what your surface should become, and you own that surface. Run it before you tell Jillienne it is ready.
It asks you	For every flag whose default your branch changed: should this be the default, and which variant? Design-bar controls are listed separately, because those never ship as defaults.
It does	Compares your branch with `main` and lists every candidate. Groups them by the Prototypes row they reach; a group that is not yours needs that owner's yes. Edits the flag defaults, runs the checks, commits on your branch. Reminds you to archive the Refinement row after the merge.
It never does	Merge anything. Add a Prototypes row. Change the default dashboard version. All three are Jillienne's.

Jillienne's — recognise, don't run

ROUTINE	WHAT IT IS FOR
`/ship-to-main`	Merging a finished branch into `main`. The only way anything reaches the product. It refuses a branch whose owner has not run `/promote-to-prototype`.
`/promote-to-testing`	Freezing a branch onto the separate user-test site and handing back participant and moderator links. A frozen link does not move when `main` does.
`/retire-from-refinement`	Closing a Refinement row properly when the work was promoted or declined: deciding what happens to the branch and writing the record of why. Different from Archive on the kebab, which only hides.
`/archive-a-feature`	Retiring a component or variant without deleting it, and writing the Archive-section row that says how to bring it back.
`/dev-handoff-notes`	Writing the handoff page engineering reads on the Development board. She runs it with you, because it asks for your words.

12 The morning auto-rebase

You can have Claude rebase your branch for you every weekday morning, so section 04 becomes something that happens while you get coffee. Claude Code has scheduled tasks: you describe the job

once, and it runs it in your project folder on a schedule, in a fresh session, and sends you a note when it is done.

Two things to know before you set it up:

- **It runs while the Claude app is open.** If your laptop was closed at 8:30, it runs the next time you open the app. In practice that means "when I start my day".
- **It works on whichever branch is checked out** in your `xcel-ux-dashboard` folder. That is the branch you were on when you closed the app yesterday, which is almost always the one you want.

Set it up once. Open the Code tab on the project and paste this, exactly:

```
Create a scheduled task called morning-rebase that runs every weekday at 8:30 in
this project folder. Each run, do the following and nothing else:

1. Report the current branch. If it is `main`, or there is no branch checked out,
stop and say so.
2. If there are uncommitted changes, stop and say so. Do not stash them and do not
commit them.
3. Fetch origin. If the branch is zero commits behind origin/main, report "already
current" and stop.
4. Create a local backup branch named backup/‹branch›/‹today's date› pointing at
the current commit. Do not push it.
5. Rebase the branch onto origin/main. If any file conflicts, abort the rebase so
the branch is exactly as it was, and report which files conflicted and in which
commit.
6. If the rebase succeeded, run `npm run build` and then `npx vitest run`. If
either fails, do not push. Report what failed; say that the local branch is
rebased but origin still has yesterday's version.
7. If both passed, push the branch to origin with `--force-with-lease`. Never push
main. Never push without the lease.
8. Finish with a six-line summary: the branch; how many commits behind it was;
whether there were conflicts; whether build and tests passed; whether it pushed;
and the Netlify review link for the branch.

Never run `git switch main`, never edit files, never resolve a conflict yourself,
and never delete anything.
```

Claude confirms the schedule and the task name. Say yes.

What you see each morning. A short note in the Claude app: rebased and pushed, or stopped and why. If it says it stopped on a conflict, do section 04 step 5 by hand. If it says checks failed, open the Code tab and say *"Show me what failed in the morning rebase"*.

Useful sentences.

YOU WANT TO	SAY
See what is scheduled	*"List my scheduled tasks."*
Run it right now instead of waiting	*"Run the morning-rebase task now."*
Change the time	*"Change morning-rebase to run at 7:45 on weekdays."*
Pause it for a week	*"Disable morning-rebase until next Monday."*
Clean up the backups it made	*"Delete my local backup/ branches older than two weeks."*

THE WORST IT CAN DO

It never touches `main`, never writes code, and never pushes anything that did not build and pass. The worst morning it can give you is "stopped, here is why".

13 Glossary

TERM	MEANING HERE
Claude Code	The Code tab in the Claude desktop app. A chat that can read and change the files in a folder, and run the project's checks for you.
Repository, repo	The project's whole history and files, on GitHub and copied to your computer. Ours is `jillmoreno/xcel-ux-dashboard`.
Clone	Copying the repository to your computer the first time.
`main`	The branch that is the product. Jillienne's.
Branch	Your own line of work, named `feat/…`. Netlify builds every branch at its own address.
Commit	A saved snapshot with a message. Several per day is normal.
Push	Sending commits to GitHub. Netlify builds after every push.
Fetch	Asking GitHub what has changed, without changing your files.
Rebase	Replaying your commits on top of today's `main`. Done on your own branch, as often as you like.
Force push with lease	The push you need after a rebase. Safe on your branch; it refuses if someone else pushed there first. Never on `main`.
Conflict	`main` and your commit changed the same lines. You pick which wins.
Merge, ship to main	Jillienne putting a branch into `main` with `/ship-to-main`. The only way anything reaches `main`.
Feature flag	A named switch in the code that picks between the old and new version of something. On by default on your branch; Jillienne decides the default on `main`.
Sibling copy	Copying a component to a V2 file instead of editing it, so two people never edit the same file. `/promote-component` does this.
Scope, owner	Each Prototypes row is a scope with an owner who decides what it should be: Compass is Anjani's, Atlas Learning is Eric's,

TERM	MEANING HERE
	Invite & Purchase is Afsana's and Eric's, Admin is Cody's, End-to-End is Jillienne's.
Chunk	One piece of work, one branch, one Refinement row. Chunks of the same feature group together on the board.
Feature (on Refinement)	What a chunk is part of, and the heading its row sits under. Free text, with the board's existing features offered as you type.
Share (was promote to refinement)	Putting one chunk of your branch's work on the Refinement board for discussion. Anyone, any time.
Promote to prototype	The owner recording which flags are their surface's defaults, on their branch. Not a release.
Branch build, review link	Your branch as a website: `https://‹branch›--ux-design-xceldashboard.netlify.app/…?demo=1`.
`?demo=1`	Turns on the demo controls bar, so a reviewer can change learner, progress and tier.
Archive (Refinement)	Hiding a row from the board, reversibly, with a stamp of who and when.
Archive (nav section)	The list of things removed from the project and how to bring each back. Different thing, same word.
Explorations	Standalone HTML documents on the Hub. Not the product.
Pull request, PR	GitHub's page for a branch that is proposed for `main`. Jillienne opens and merges them.

Part of the XCEL UX Dashboard. The working copy of this guide is the Claude doc *Designer Workflow on the Hub*, which Jillienne edits; this page is what the Hub serves. The reasoning behind every decision described here is in the repo's `CLAUDE.md`. The longer walkthrough, with pictures, is the Contributing guide; stakeholders get How to read this dashboard.